



Wrocław
University
of Science
and Technology

Software Engineering

UML-based modeling of structure and the C4 model

Marcin Jodłowiec

November 12nd & November 19th, 2025

Agenda

- The C4 Model
 - Context
 - Containers
 - Components
 - Code
- UML-based structural modeling
 - Class diagram
 - Object diagram
 - Package diagram
 - Composite structure diagram
 - Component diagram
 - Deployment diagram
- Summary

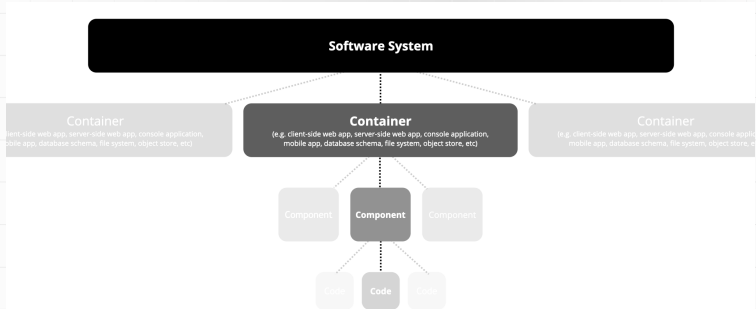
The C4 Model

C4 Model

- ▶ Introduced by Simon Brown (2006–2011) as a modern alternative to UML.
- ▶ Useful for distributed architectures.
- ▶ Defines four hierarchical levels of abstraction:
 - ▶ **Context** – system and its external actors.
 - ▶ **Container** – applications, databases, and services within the system.
 - ▶ **Component** – internal building blocks of each container.
 - ▶ **Class (Code)** – detailed code-level structures (optional).
- ▶ Lightweight, tool-friendly, and widely adopted in modern software architecture.

C4 Model

Abstractions



A **software system** is made up of one or more **containers** (applications and data stores), each of which contains one or more **components**, which in turn are implemented by one or more **code** elements (classes, interfaces, objects, functions, etc).

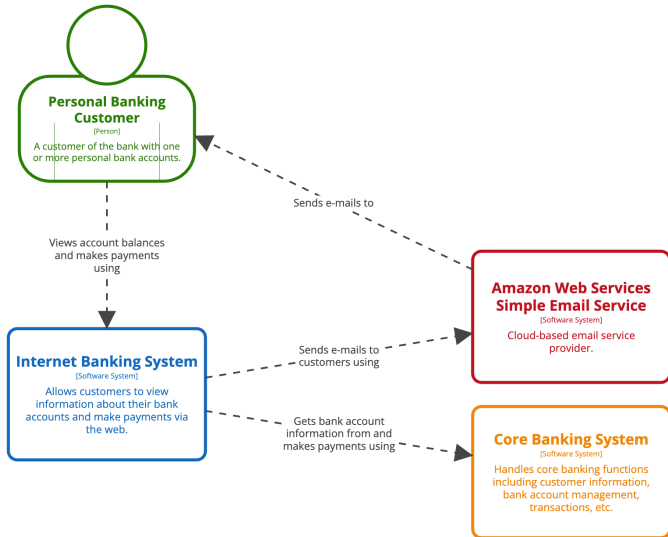
The C4 Model

Software System (C4 – Level 1 – Context)

- ▶ **Highest level of abstraction** – represents the entire software solution that delivers value to its users (human or not).
- ▶ **Team ownership boundary** – typically what a single development team *owns, builds, and deploys as a whole* (often mapped to one source code repository).
- ▶ **Context of dependencies** – the system interacts with other software systems and external services, forming its **system context**.

C4 Model

Context



System Context View: Internet Banking System

The system context diagram for a fictional Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

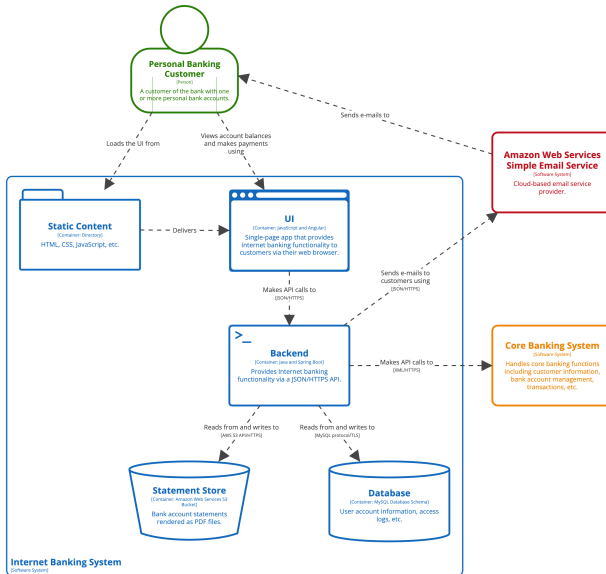
The C4 Model

Container (C4 – Level 2)

- ▶ **It is not Docker!** In the C4 model, a *container* represents an application or data store that must be running for the system to work.
- ▶ **Runtime boundary** – it's a deployable unit that executes code or stores data (e.g. web app, mobile app, database, serverless function).
- ▶ **Examples:**
 - ▶ Server-side web app, client-side SPA, mobile or desktop app.
 - ▶ Database, file store, or blob storage.
 - ▶ Serverless function or background process.
- ▶ **Team perspective** – each container usually represents a separately deployable runtime environment that can be modeled independently in the architecture.

C4 Model

Containers



Container View: Internet Banking System

The container diagram for the Internet Banking System | Simon Brown | c4model.com | License: CC BY 4.0

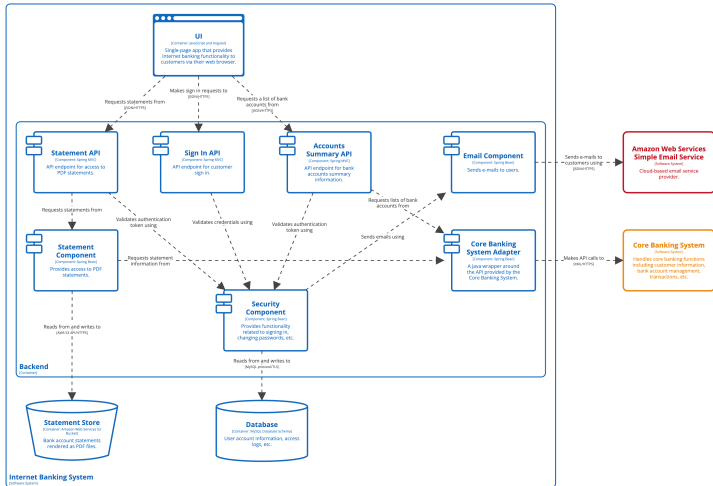
The C4 Model

Component (C4 – Level 3)

- ▶ **Logical grouping of related functionality** – encapsulated behind a clear interface; typically several classes, modules, or functions working together.
- ▶ **Not separately deployable** – all components within a container run in the same process; deployment happens at the container level.
- ▶ **Bridges code and architecture** – abstracts from low-level classes and files to higher-level functional units (e.g. controllers, services, repositories).
- ▶ **Language-agnostic concept:**
 - ▶ OOP langs: collection of classes and interfaces.
 - ▶ JavaScript: module composed of functions and objects.
 - ▶ Functional languages: logical module grouping related functions and types.

C4 Model

Components



Component View: Internet Banking System - Backend

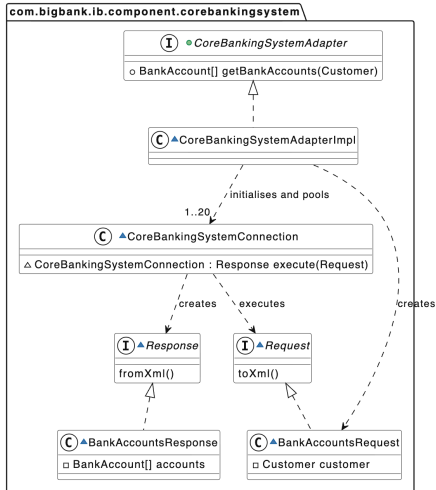
The component diagram for the Internet Banking System Backend | Simon Brown | olnmodel.com | License: CC BY 4.0

Code (C4 – Level 4)

- ▶ **Lowest level of detail** – shows how a component is implemented using code elements of a specific programming language.
- ▶ **Building blocks:** classes, interfaces, functions, objects, enums, modules, etc.
- ▶ **Used selectively** – detailed diagrams are only created for complex or critical components where code structure matters for understanding.

C4 Model

Code



Code View: Internet Banking System - Backend - Core Banking System Adapter

A summary of the implementation details for the Core Banking System Adapter component | Simon Brown | c4model.com | License: CC BY 4.0

UML-based structural modeling

Introduction

Why Structural Modeling?

- ▶ Structural models describe the *static architecture* of a software system: how it is organized in terms of elements, relationships, and dependencies.
- ▶ **Static vs dynamic** part of software system:
 - ▶ **Static** – structure, components, classes, deployment.
 - ▶ **Dynamic** – behavior, interactions, state changes.
- ▶ **Role in software engineering:**
 - ▶ Documentation and architectural communication.
 - ▶ Reverse engineering and understanding legacy systems.
 - ▶ Bridging domain analysis with design and implementation.
 - ▶ Model-based / model-driven engineering.

UML Structure Diagrams

Overview

Diagram

A UML diagram is a partial **graphical representation** (view) of a **model** of a system under design, implementation, or already in existence. UML diagram contains graphical elements (symbols) - UML nodes connected with edges (also known as paths or flows) - that represent elements in the UML model of the designed system.

Structural diagrams:

- ▶ Show the **static structure** of a system and its parts.
- ▶ Represent concepts at different abstraction and implementation levels.
- ▶ Do **not** describe time-related or dynamic behavior.
- ▶ May show relationships to behaviors of the classifiers.

Core Structure Diagrams

- ▶ **Class Diagram** – structure of designed system, subsystem or component as related classes and interfaces, with features, constraints and relationships.
Elements: class, interface, feature, association, generalization, dependency.
- ▶ **Object Diagram** – instance-level class diagram; shows concrete objects and links (snapshot of system state).
Elements: instance specification, slot, link.
- ▶ **Package Diagram** – organizes elements into packages and shows their dependencies.
Elements: package, dependency, import, merge.
- ▶ **Composite structure diagram** – internal structure of a classifier
Elements: classifiers, roles, parts, connectors.
- ▶ **Component Diagram** – components and dependencies between them
Elements: component, interface, port, connector, artifact, realization.
- ▶ **Deployment diagram** – Shows physical architecture of the system as deployment (distribution) of software artifacts to deployment targets.
Elements: devices, execution environments, nodes, artifacts, manifestations, components

Class Diagram

Basic info

- ▶ Shows the **logical structure** of the system: classes, their attributes, operations, and relationships.
- ▶ Each class is a **classifier**, i.e. a template for objects that share structure and behavior.
- ▶ Often used to:
 - ▶ derive OOP code structure,
 - ▶ analyze domain models (conceptual view),
 - ▶ analyze information needs,
 - ▶ visualize architecture of subsystems.
- ▶ Level of abstraction: between conceptual domain model and implementation-level code.

Class Diagram

Classifiers

- ▶ **Classifier** – an abstract element defining a set of instances sharing the same features.
 - ▶ Examples: class, interface, enumeration, data type, component.
- ▶ **Class notation:**

ClassName
attribute_1 : Type
operation(parameter : Type) : ReturnType

- ▶ Classes may have:
 - ▶ *visibility* (+ public, – private, # protected),
 - ▶ *stereotypes* («entity», «control», etc.),

Class Diagram

Relationships

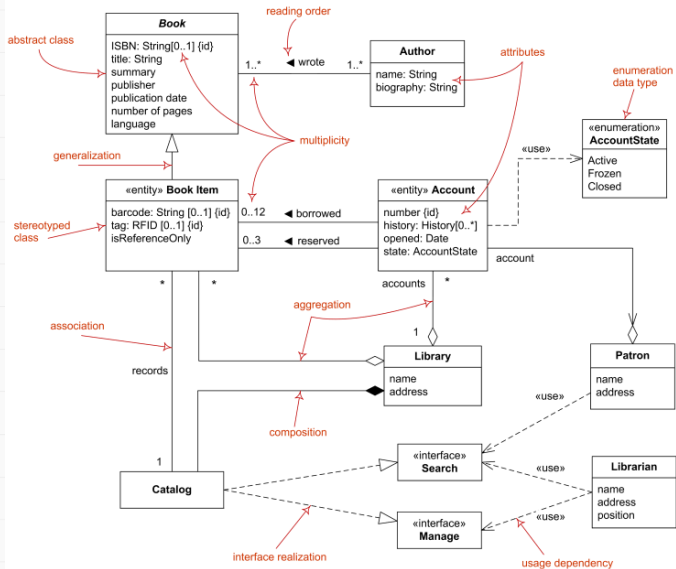
- ▶ **Association** – structural connection between classifiers (e.g. Student–Course).
 - ▶ may be named, navigable, have roles and multiplicities (e.g. 1..*, 0..1).
- ▶ **Aggregation** (empty diamond) – “whole-part” relationship with weak ownership.
- ▶ **Composition** (filled diamond) – strong ownership; parts are destroyed with the whole (*existence dependency*).
- ▶ **Generalization** (triangle arrow) – inheritance relation; subclass specializes superclass.
- ▶ **Dependency** (dashed arrow) – one element uses or depends on another.

Class Diagram

Multiplicity and Association Details

- ▶ **Multiplicity** – number of instances related to another class.
 - ▶ $\infty \neq *$
 - ▶ asterisk means that the upper bound is not specified (unlimited in the sense of the model, not infinite in mathematics)
 - ▶ Examples: 1, 0..1, *, 1..*, 2..5
- ▶ **Arity** – number of association ends (typically 2).
- ▶ **Role names** – clarify semantic meaning of association ends.
- ▶ **Association class** – allows adding attributes or operations to a relationship.
- ▶ Enables modeling of real-world constraints (e.g. “each student may be enrolled in many courses, each enrollment has a grade”).

Class diagram



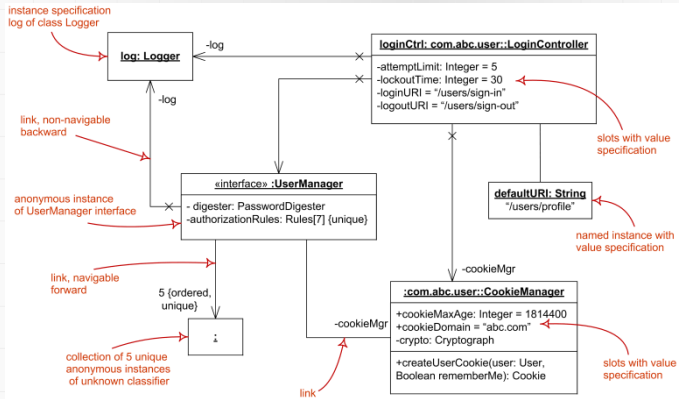
Source: uml-diagrams.org, great UML 2.5 standard compendium

Object Diagram

Basic info

- ▶ **Instance-level representation** – shows concrete objects and their relationships at a specific moment in time (a snapshot of the system state).
- ▶ **Complements Class Diagram** – while class diagrams describe templates, object diagrams show actual instances with specific attribute values.
- ▶ **Use cases:**
 - ▶ Visualizing complex object interactions in specific scenarios.
 - ▶ Documenting state of the system at critical points.
 - ▶ Testing and validating object relationships.
 - ▶ Analyzing object creation and lifecycle patterns.
- ▶ **Elements:**
 - ▶ *Instance specification* – concrete object with a name and class.
 - ▶ *Slot* – attribute value of a specific instance.
 - ▶ *Link* – connection between object instances (instance of association).

Object diagram



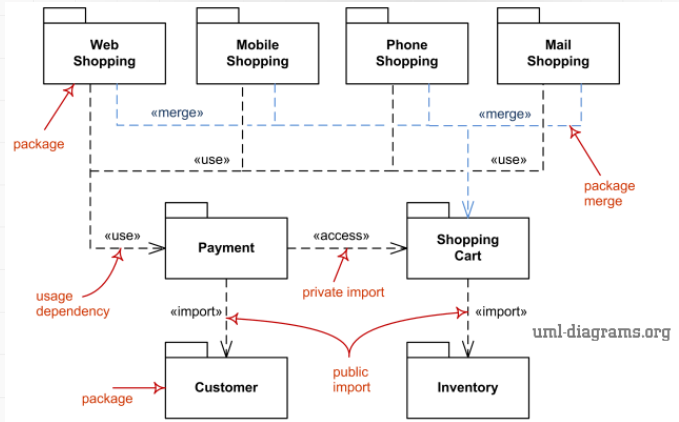
Source: uml-diagrams.org, great UML 2.5 standard compendium

Package Diagram

Basic info

- ▶ **Organizational structure** – shows how model elements are grouped into packages and the dependencies between packages.
- ▶ **Package concept** – a namespace that groups related model elements (classes, interfaces, use cases, other packages).
- ▶ **Use cases:**
 - ▶ Organizing large systems into manageable subsystems/modules.
 - ▶ Visualizing layered architecture and dependencies.
 - ▶ Managing complexity through hierarchical decomposition.
 - ▶ Analyzing coupling between different parts of the system.
- ▶ **Key elements:**
 - ▶ *Package* – container grouping related elements.
 - ▶ *Dependency* – relationship showing one package requires another.
 - ▶ *Import/Access* – mechanism for using elements from other packages.
 - ▶ *Merge* – combining contents of multiple packages.

Package diagram



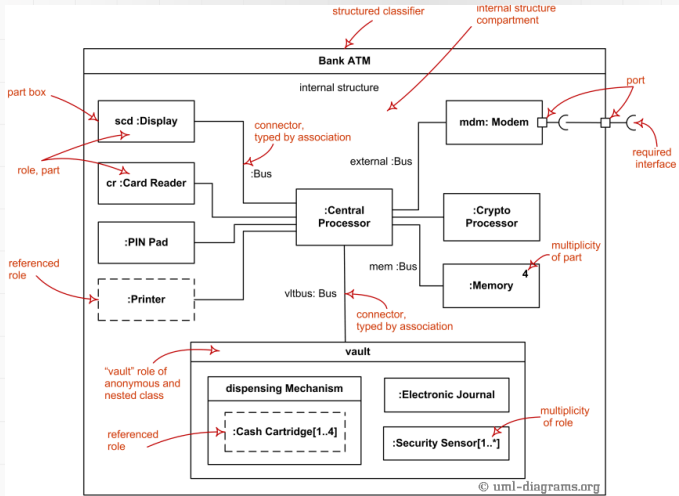
Source: uml-diagrams.org, great UML 2.5 standard compendium

Composite Structure Diagram

Basic info

- ▶ **Internal structure view** – shows the internal organization of a classifier (class, component, or collaboration) and how its parts interact.
- ▶ **Runtime configuration** – describes how instances are structured and connected at runtime, complementing class diagrams with collaboration details.
- ▶ **Use cases:**
 - ▶ Modeling complex internal architectures of components or subsystems.
 - ▶ Showing collaboration patterns and communication paths.
 - ▶ Defining ports and interfaces for component-based design.
 - ▶ Describing runtime assembly and wiring of parts.
- ▶ **Key elements:**
 - ▶ *Part* – internal element representing a role or instance within the classifier.
 - ▶ *Port* – interaction point between classifier and its environment.
 - ▶ *Connector* – communication link between parts or ports.
 - ▶ *Interface* – contract specifying provided/required operations.

Composite structure diagram



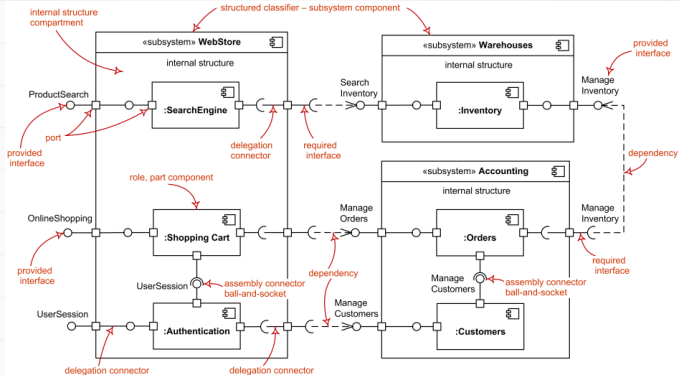
Source: uml-diagrams.org, great UML 2.5 standard compendium

Component Diagram

Basic info

- ▶ **High-level structural view** – shows components, their provided/required interfaces, and dependencies between components.
- ▶ **Component concept** – a modular, replaceable, and reusable part of a system that encapsulates implementation and exposes interfaces.
- ▶ **Use cases:**
 - ▶ Modeling system architecture at component level.
 - ▶ Defining clear interfaces and contracts between subsystems.
 - ▶ Supporting component-based development and reuse.
 - ▶ Visualizing deployment and integration architecture.
- ▶ **Key elements:**
 - ▶ *Component* – modular unit with well-defined interfaces.
 - ▶ *Interface* – contract specifying provided or required operations.
 - ▶ *Port* – interaction point where component exposes interfaces.
 - ▶ *Connector* – communication path between components.
 - ▶ *Dependency* – relationship showing component dependencies.

Component diagram



Source: uml-diagrams.org, great UML 2.5 standard compendium

Component Diagram

Interface vs Port

Intuitive analogy – network printer as a component:

- ▶ **Interface** = "communication language" / API contract
 - ▶ Defines WHAT operations: print(), checkToner(), getStatus()
 - ▶ This is the CONTRACT (what you can communicate)
- ▶ **Port** = connection point
 - ▶ Specifies WHERE and HOW the interface is exposed
 - ▶ Examples: USB, Wi-Fi, Ethernet, Bluetooth
 - ▶ One interface can be exposed through multiple ports

Engineering practice examples:

- ▶ **RESTful Service:**
 - ▶ Interface: IOrderService (createOrder, getOrder, cancelOrder)
 - ▶ Ports: HTTP/REST endpoint (port 8080), gRPC endpoint (port 9090)
- ▶ **Database:**
 - ▶ Interface: IDataAccess (CRUD operations)
 - ▶ Ports: TCP/IP (port 5432), Unix socket, connection pool

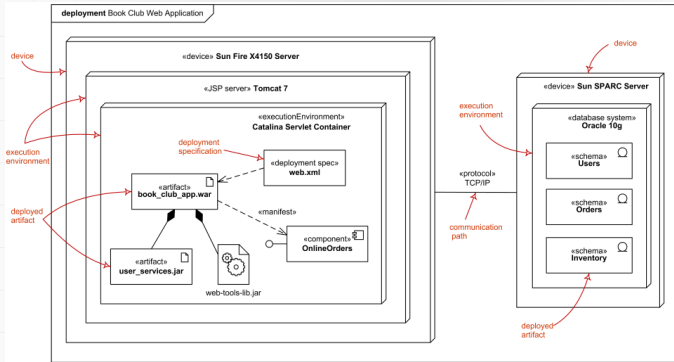
Interface = "WHAT you offer" (contract), Port = "HOW to access it" (mechanism)

Deployment Diagram

Basic info

- ▶ **Physical architecture view** – shows how software artifacts are deployed onto hardware nodes and execution environments.
- ▶ **Runtime infrastructure** – models the physical deployment of components, applications, and data to computing resources.
- ▶ **Use cases:**
 - ▶ Visualizing system topology and hardware configuration.
 - ▶ Planning deployment strategy and infrastructure requirements.
 - ▶ Documenting distribution of software across physical nodes.
- ▶ **Key elements:**
 - ▶ *Node* – physical or virtual computing resource.
 - ▶ *Artifact* – physical piece of information (executable, library, database).
 - ▶ *Deployment specification* – configuration parameters.
 - ▶ *Communication path* – network connection between nodes.

Deployment diagram



Source: uml-diagrams.org, great UML 2.5 standard compendium

Summary

C4 Model vs UML Structure Diagrams

Comparison

C4 Model

- ▶ **Simple & focused** – 4 levels of abstraction
- ▶ **Communication-oriented** – easy for stakeholders
- ▶ **Top-down approach** – from context to code
- ▶ **Modern architectures** – microservices, distributed systems
- ▶ **Lightweight notation** – minimal symbols

UML Structure Diagrams

- ▶ **Comprehensive** – many diagram types
- ▶ **Standardized** – UML 2.5.4 (OMG)
- ▶ **Detail-rich** – precise technical specifications
- ▶ **Object-oriented focus** – classes, relationships, etc.
- ▶ **Formalized notation** – extensive metamodel

When to use:

- ▶ **C4:** High-level architecture documentation, stakeholder communication, system context
- ▶ **UML:** Detailed design, code generation, formal modeling, reverse engineering



It's all for today!